Q6 – Deadlock and Inter-Thread Communication

Part A – Deadlock

```
class Account {
    final int id; int balance;
    Account(int id, int b) { this.id = id; balance = b; }
}

public class Wallets {
    // Deadlock-prone
    static void transferBad(Account from, Account to, int amt) {
        synchronized (from) {
            try { Thread.sleep(50); } catch (InterruptedException e)
{}
            synchronized (to) { from.balance -= amt; to.balance +=
amt; }
        }
    }
    // Deadlock-free: consistent lock ordering by id
    static void transferSafe(Account from, Account to, int amt) {
        Account first = from.id < to.id ? from : to;
        Account second = from.id < to.id ? to : from;
        synchronized (first) {
            synchronized (second) { from.balance -= amt; to.balance +=
amt; }
        }
    }
    public static void main(String[] args) throws Exception {
        Account a = new Account(1, 1000), b = new Account(2, 1000);
        Thread t1 = new Thread(() -> transferBad(a, b, 100));   //
locks a then b
        Thread t2 = new Thread(() -> transferBad(b, a, 50));    //
locks b then a
        t1.start(); t2.start(); t1.join(); t2.join();           //
hangs → deadlock
        // switch both to transferSafe → completes
    }
}
```

Student B

- **Reliable deadlock:** transfer A→B and B→A at the same time with a `sleep` between the two locks, as above. Confirm with `jstack <pid>`, which reports "Found one Java-level deadlock".
- **Four conditions present:** (1) *mutual exclusion*: a monitor is held by one thread at a time; (2) *hold and wait*: each thread holds one account lock while waiting for the other;

(3) *no preemption*: a monitor can't be forcibly taken; (4) *circular wait*: T1 waits for T2's lock and T2 waits for T1's.

- **Prevention:** always lock the account with the smaller id first (code above).
- **Why it works:** it breaks circular wait. All threads acquire locks in one global order, so a cycle in the wait-for graph is impossible. (Take care with `from == to`, which should be handled separately.)

Part B – Producer–consumer (Complete the Code)

```
public class TokenCounter {
    private final int[] buffer = new int[5];
    private int count = 0, in = 0, out = 0;

    public synchronized void put(int token) throws
InterruptedException {
        while (count == buffer.length) wait();        // TODO 1
        buffer[in] = token;
        in = (in + 1) % buffer.length;
        count++;
        notifyAll();
    }

    public synchronized int take() throws InterruptedException {
        while (count == 0) wait();                    // TODO 2
        int token = buffer[out];
        out = (out + 1) % buffer.length;
        count--;
        notifyAll();
        return token;
    }
    // main exactly as given
}
```

Expected output: `Sum of tokens collected: 210 (expected 210)` (1+2+...+20 = 210).

Change `while` to `if`: the program may hang, throw `ArrayIndexOutOfBounds`, produce a wrong sum, or read stale/duplicate tokens in some runs. With `if`, a woken thread doesn't re-check the condition. After `notifyAll()`, another consumer may have already taken the token (or another producer filled the slot), so the woken thread proceeds with `count == 0` (or `== 5`). Run it many times and record.

Student A

- **`wait()` in a synchronized block:** it must own the monitor (otherwise `IllegalMonitorStateException`), and `wait()` atomically releases it while sleeping. **In a loop:** because of spurious wakeups and because the condition may have changed again by the time the thread reacquires the lock.
- **`notify()` vs `notifyAll()`:** with several consumers and producers waiting on the same monitor, `notify()` may wake the wrong kind of thread (e.g. a producer when the

buffer is full), which goes back to waiting while the right thread never wakes, causing a lost wakeup / stall. `notifyAll()` wakes all, and each re-checks its condition in the `while` loop. (Here one producer + 2 consumers: `notify()` can still stall if it wakes a consumer when another is needed in some designs. Use `notifyAll()` to be safe.)

- **Counter size over time:** log `count` inside `put/take` (inside the lock) and show that it stays in 0..5.
- **`sleep()` vs `wait()`:** `sleep` is a static `Thread` method that pauses for a time and **keeps** any locks. `wait` is an `Object` method, must be called while holding the monitor, **releases** the monitor and waits until `notify/notifyAll`/timeout/interrupt.

Joint conclusion

Both involve threads blocked on a condition held by other threads. In deadlock the waiting is circular and nobody progresses. In producer–consumer the waiting is mutually dependent but acyclic: producers wait for space, consumers wait for items. Deadlock is prevented by breaking circular wait (lock ordering). Producer–consumer makes progress because every state change calls `notifyAll()` and waiters re-check the condition, so a producer's put always makes consumers eligible and vice versa.

Alternative

```
// ArrayBlockingQueue
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(5);
q.put(token);   int t = q.take();

// tryLock with timeout
if (a.lock.tryLock(100, TimeUnit.MILLISECONDS)) {
    try {
        if (b.lock.tryLock(100, TimeUnit.MILLISECONDS)) {
            try { /* transfer */ } finally { b.lock.unlock(); }
        } else { /* back off and retry */ }
    } finally { a.lock.unlock(); }
}
```

- **Time:** `put/take`: O(1) either way (circular buffer). The blocking queue has internal lock/condition overhead but is well-tested. `tryLock` transfer: O(1) per attempt, but retries add time under contention (possible livelock without randomized backoff). Lock ordering: O(1), with the cost of waiting only.
- **Space:** buffer O(5)=O(capacity) in both. Blocking queue adds a lock and two conditions. `tryLock` needs one `ReentrantLock` per account, O(#accounts).
- **Less error-prone:** `ArrayBlockingQueue` (no manual wait/notify, no lost wakeups). For transfers, lock ordering is simpler and deterministic. `tryLock` needs retry logic and failure handling.